

Voy al grano. Te resumo cómo encaja todo y dónde están los puntos clave/interesantes.

Arquitectura general (bien planteada)

Tienes un pipeline bastante limpio:

1. **XML → estructura interna**
 - ReportItemLoadEngine convierte XML a objetos (GroupItem, Band*)
 - Muy desacoplado → 👍 extensible
2. **Modelo de datos**
 - ModelReport carga \$data
3. **Template**
 - ExportTemplate gestiona datasets + config
4. **Render**
 - PDF → PDFTemplate
 - CSV → CSVTemplate

👉 Separación perfecta: **estructura / datos / render**

Lo más potente que tienes

1. Sistema de grupos recursivo

```
public BandDetail|GroupItem $detail;
```

Esto es clave:

- Permite **grupos anidados sin límite**
- Y se usa elegantemente:

```
if ($this->detail instanceof GroupItem)
```

👉 Esto es lo que te permite hacer:

- país → familia → producto
sin hacks

2. Ruptura de secuencia (core del sistema)

```
hasFieldRupture()
```

- ✓ Detecta cambio de valor
- ✓ Mantiene estado interno (`fieldValue`)

Y en render:

```
$hasRupture = $detail->hasFieldRupture($row, true);
```

👉 Esto es literalmente el motor de agrupación.

3. Render inteligente con control de página

Muy buen detalle:

```
$remaining = $this->pagePosition($position) - $footerHeight;  
$required = $detail->height + $detHeaderHeight;
```

- ✓ Evitas cortar bloques
- ✓ Re-renderizas headers al saltar página

👉 Esto no es trivial → está bien resuelto.

4. Widgets desacoplados

Especialmente potente:

`fieldname` puede ser:

- `propiedad`
- `objeto.propiedad`
- `función()`
- `array[index]`

👉 Esto te da un mini lenguaje dentro del XML.

5. Cálculos en footer (muy buena idea)

```
$footer?->calculate($data);
```

- `WidgetCalculated`

👉 Permite:

- sumas por grupo
- conteos
- etc.

⚠ Cosas a vigilar (aquí está lo interesante)

1. `WidgetCalculated::avg` está mal conceptualmente

```
return ($this->value + $newValue) / 2;
```

❌ Esto NO es una media real

👉 Es una media acumulativa incorrecta

Deberías:

- guardar suma + count
- calcular al final

2. Estado mutable en `BandDetail`

```
public ?string $fieldValue = null;
```

Problema:

- estado global durante render
- si reutilizas instancia → puede romper

👉 Funciona, pero es frágil si cambias flujo

3. Header/Footer con `type main/second`

```
$bands[BandItem::BAND_TYPE_SECOND]
```

Idea buena pero:

- poco explícita en XML
- difícil de mantener mentalmente

4. CSV ignora agrupaciones

```
$detail = $group->getDetail();
```

 No usa:

- headers
- rupturas
- subgrupos

 CSV $\neq$ PDF (comportamiento distinto)

5. **forceNewPage** no se usa en render

Lo defines en:

```
public bool $forceNewPage;
```

Pero no lo veo aplicado en:

- `renderHeader`
- `renderFooter`

 Parece feature a medio implementar

Conclusión rápida

Tu sistema es:

-  Muy flexible (XML + widgets + grupos)
-  Bien desacoplado
-  Potente para reporting complejo

Pero:

- ⚠ Hay pequeñas incoherencias (avg, CSV, flags no usados)
- ⚠ Algo de estado interno que puede escalar mal

🗨 Tema para comentar contigo

Te dejo una pregunta clave (aquí está lo interesante):

👉 ¿Quieres que el sistema sea **tipo JasperReports (muy declarativo)** o seguir con esta mezcla de **XML + lógica PHP implícita**?

Porque ahora mismo estás en un punto intermedio:

- Muy flexible
- Pero empieza a ser difícil de razonar en ciertos casos

Si quieres, en el siguiente paso podemos:

- revisar tu caso concreto (ruptura por país/familia)
- o proponer mejoras de diseño (nivel serio)

Tú dirás 👍

comencé con RosPdf por su velocidad. Pero estoy viendo mucho potencial a mPdf. Merecería la pena el cambio?

Pensó durante 17s

Sí, **puede merecer la pena**, pero no como sustitución total inmediata.

Viendo tu arquitectura, RosPdf/Cezpdf encaja muy bien con lo que ya tienes: trabajas con bandas, posiciones, alturas y control manual de página, y eso casa con un motor de dibujo directo como R&OS PDF, que sigue siendo una librería ligera para generar PDF sin módulos extra y con helper Cezpdf para tablas, fondos y paginación. Además, el proyecto sigue vivo: su repositorio oficial tuvo una release reciente (0.12.67, 20 de abril de 2026).

El cambio a **mPDF** gana mucho sentido cuando quieres que el informe deje de ser “pintado a mano” y pase a ser más **declarativo/HTML**: mPDF genera PDF desde HTML UTF-8, soporta imágenes inline, cabeceras y pies HTML, CSS y un manejo bastante más cómodo para maquetaciones ricas y documentos multilingüaje.

La pega es la que ya intuyes: **mPDF suele ser más lento**. Su propia documentación lo dice claramente: es más lento que las librerías base y las tablas grandes son una de las mayores cargas; eso sí, ofrece opciones para aliviar parte del coste, como **simpleTables**, que puede reducir memoria y mejorar rendimiento alrededor de un 30% en tablas grandes, y **packTableData**, que ahorra memoria a costa de más tiempo de proceso.

Por cómo tienes montado **ExtendedReport**, yo no haría un “switch” completo ahora. Haría esto a nivel estratégico:

- **Mantener RosPdf** para informes tabulares, estadísticos, rápidos y muy controlados.

- **Añadir mPDF como segundo backend** para informes más visuales, fichas, documentos con bloques ricos, HTML, estilos complejos, logos, imágenes, textos largos y futuras gráficas.

Esto además encaja muy bien con tu diseño actual, porque ya tienes una capa base

`ExportTemplate` y luego renderizadores concretos (`PDFTemplate`, `CSVTemplate`).

Esa separación hace bastante natural crear otro renderer tipo `MPDFTemplate` sin romper el modelo de datos ni la carga XML.

Mi recomendación real sería esta:

Sí merece la pena introducir mPDF, pero no reemplazar RosPdf de golpe.

Lo usaría como **motor alternativo**. RosPdf seguiría siendo tu opción de alto rendimiento; mPDF sería tu opción de alta expresividad.

La decisión práctica depende de cuál sea tu objetivo principal:

- Si priorizas **velocidad, simplicidad y control milimétrico**, seguiría con RosPdf.
- Si priorizas **evolución visual, plantillas más cómodas y futuras gráficas/bloques complejos**, sí apostaría por mPDF.

En tu caso, con lo que ya estás construyendo, yo haría una migración **parcial y progresiva**, no una sustitución completa.

Si te parece, el siguiente paso útil es compararlos **contra tu caso real**: informes estadísticos con rupturas, subtotales, cabeceras repetidas y posibles gráficas. Ahí se ve enseguida si mPDF compensa o no.

Pensó durante 4s

Sí. Viéndolo así, el tema ya no es solo **RosPdf vs mPDF**, sino **qué modelo de definición quieres para ExtendedReport**.

Y siendo sincero: para lo que quieres ahora, **el XML empieza a quedarse corto o, mejor dicho, se vuelve incómodo**.

Tu arquitectura actual está muy bien para un motor clásico de reporting:

- `ExportTemplate` ya separa datos y renderizado.
- Tienes renderizadores distintos para PDF y CSV.
- Los grupos y bandas permiten composición bastante potente.

Pero el coste es claro: el XML obliga a describir todo de forma muy rígida. Eso funciona para informes tabulares puros, pero empieza a pesar cuando quieres:

- una salida HTML visual
- interacción en pantalla
- reutilizar plantilla entre vista web y PDF
- añadir gráficos o bloques más ricos
- abrir la puerta a XLS/XLSX con otra capa de salida

Mi opinión

Yo no tiraré ExtendedReport, pero sí cambiaría el centro del sistema:

Antes

XML = definición principal del informe

Después

Datos + plantilla vista = definición principal del informe

Es decir:

- el **modelo** sigue cargando y preparando datos
- la **plantilla Twig** pasa a ser la forma natural de presentar
- luego decides el destino:
 - HTML en pantalla
 - HTML -> PDF con mPDF
 - exportación XLS/XLSX
 - CSV

Por qué esto te encaja más

Porque tú ya no quieres solo “imprimir informes”.

Quieres un sistema de **presentación multicanal**.

Y el XML de bandas no es bueno para eso.

Es bueno para imprimir con coordenadas, pero malo para evolucionar hacia UI.

En cambio Twig te da:

- una sola plantilla mentalmente más fácil
- reutilización entre web y PDF
- más rapidez para iterar
- posibilidad de meter condicionales, bloques, parciales
- integración futura con gráficos JS en pantalla y gráficos estáticos para PDF

Qué haría yo

Haría una transición en 3 capas:

1. Mantener el modelo de datos

`ModelReport` o una evolución suya sigue teniendo mucho sentido.

Pero intentaría que el modelo no piense en PDF, sino en:

- dataset principal
- metadatos
- totales
- agrupaciones
- config de render

2. Crear “renderers” de salida

Algo así conceptualmente:

- `HtmlReportRenderer`
- `PdfReportRendererMpdf`
- `CsvReportRenderer`

- `XlsxReportRenderer`

Esto encaja con la idea que ya tienes en `ExportTemplate`.

3. Dejar el XML como modo legado o técnico

No lo eliminaría de golpe.

Lo dejaría para:

- informes antiguos
- informes muy tabulares
- casos donde el posicionamiento exacto siga siendo útil

Pero los nuevos informes los haría ya con Twig.

Sobre XLS

Aquí además te beneficia mucho abandonar el pensamiento “banda PDF”.

Porque XLS/XLSX no debería salir de una plantilla de impresión, sino de una **estructura de datos tabular o jerárquica**.

O sea:

- si el informe es tabla → XLS directo
- si el informe es visual/resumen → HTML/PDF
- si mezcla ambas cosas → modelo común + render distinto

CSV ya te demuestra que el motor actual puede tener varios outputs, pero también deja ver sus límites: ahí realmente exportas valores del detalle, no una representación rica del informe.

Sobre mostrar datos en pantalla

Aquí es donde más claro veo el cambio.

Si quieres algo interactivo, el XML de bandas no te ayuda mucho.

Para pantalla necesitas pensar en:

- filtros
- ordenación
- paginación
- expandir grupos
- gráficos
- exportar desde la misma vista

Eso pide casi de forma natural:

- controlador
- modelo/dataset
- Twig
- JS opcional

Y luego desde esa misma base:

- “Ver en pantalla”
- “Exportar PDF”

- “Exportar XLSX”
- “Exportar CSV”

Conclusión clara

Yo haría esto:

ExtendedReport debería evolucionar de motor de informes XML/PDF a framework de informes basado en datasets + renderers.

Y en esa evolución:

- **Twig + HTML + mPDF** sería la vía principal para nuevos informes
- **RosPdf/XML** quedaría como backend legado o especializado
- **XLS/XLSX** saldría del mismo dataset, no del layout PDF
- la **vista en pantalla** pasaría a ser un objetivo de primera clase, no algo secundario

Dicho de forma simple:

sí, creo que has llegado al punto en el que el XML empieza a frenarte más de lo que te ayuda.

El cambio no sería “sustituir una librería PDF”, sino **cambiar el paradigma del plugin.**

Si quieres, en el siguiente mensaje te propongo una arquitectura concreta para esa evolución, intentando reaprovechar al máximo lo que ya tienes.

Pensó durante 7s

Sí, eso me parece una evolución **muy sensata.**

De hecho, ahí no estarías tirando el sistema, sino **aprovechando precisamente lo mejor que ya tienes:**

- el XML sigue definiendo la estructura
- el modelo sigue aportando los datos
- y añades otro renderer, igual que ya existen PDF y CSV.

Lo bueno de esa idea

Tu sistema ya está pensado como:

- **estructura** → `GroupItem, BandHeader, BandDetail, BandFooter`
- **carga del XML** → `ReportItemLoadEngine`
- **dataset** → `ModelReport + addDataset()`
- **salida concreta** → `renderer`

Así que un `ViewTemplate` o `HtmlTemplate` encaja perfectamente en la arquitectura.

En qué cambia el enfoque

No sería:

- Twig como sustituto del XML

Sino:

- XML como definición del informe

- HTML como **otra salida del mismo informe**
- Y eso tiene bastante lógica si el informe es tabular.

Qué ventaja te da

La principal: **ver el informe antes de exportarlo.**

Algo tipo:

- filtros arriba
- vista HTML central
- botones:
 - PDF
 - CSV
 - XLSX

Y en pantalla podrías hacerlo más agradable:

- tablas con estilo
- cabeceras fijas
- grupos visuales
- colapsables
- totales destacados
- colores para negativos
- scroll
- quizás ordenación o búsqueda

Todo eso sin abandonar el XML.

Lo importante técnicamente

Yo no intentaría “pintar coordenadas PDF en HTML”.

Porque en PDF tú trabajas con:

- `posX`
- `posY`
- `height`
- anchuras muy físicas

Eso en HTML no merece la pena replicarlo tal cual.

Sería más útil interpretar el XML de forma **semántica**, no geométrica.

Por ejemplo:

- `header` → bloque de cabecera
- `detail` → tabla o filas
- `footer` → fila de totales o bloque resumen
- `group/detail group` → secciones agrupadas

Es decir, usar el XML como **estructura lógica** del informe, no como layout absoluto.

Donde veo el punto delicado

El motor actual tiene lógica pensada para PDF, sobre todo:

- salto de página
- posición vertical
- header/footer second
- alturas exactas
- ruptura de secuencia ligada al render lineal

En HTML eso cambia.

Ahí yo separaría dos cosas:

1. Estructura del informe

Esto sí lo reaprovechas:

- grupos
- detalle
- columnas
- widgets
- fieldname
- cálculos

2. Comportamiento específico de salida

Esto ya no se comparte igual:

- paginación PDF
- posiciones
- segunda cabecera/pie
- forceNewPage

Mi opinión clara

Sí, un **ViewTemplate** que renderice el XML a HTML tiene mucho sentido. Probablemente es el siguiente paso más natural para ExtendedReport.

Además te permite:

- no romper compatibilidad
- no rehacer todos los informes
- abrir la puerta a una UI más útil
- reutilizar el modelo y la estructura actual

Cómo lo plantearía

Conceptualmente:

- `ExportTemplate` sigue siendo la base
- nuevo `HtmlTemplate` `extends ExportTemplate`
- nuevo método `render(): string`
- salida en Twig o incluso HTML generado desde PHP

Y luego un controlador/vista que:

- carga filtros
- ejecuta modelo
- renderiza HTML
- ofrece exportación del mismo dataset

Resumen sincero

Tu idea me parece mejor que migrar todo a Twig puro.

Porque:

- conserva tu inversión en XML
- respeta la arquitectura actual
- y añade justo lo que ahora te falta: **visualización en pantalla**

La clave sería aceptar que el HTML **no debe imitar el PDF**, sino representar el mismo informe de una forma más cómoda y bonita.

El siguiente paso útil sería definir **cómo se traduce cada banda XML a HTML** sin depender de coordenadas.

Pensó durante unos pocos segundos

Perfecto. Entonces el encaje está bastante claro y, de hecho, **refuerza** la idea de no romper ExtendedReport.

Cómo lo veo

ExtendedReport

Debe seguir siendo la **capa base y genérica**:

- cargar definición del informe
- gestionar grupos, bandas, columnas y widgets
- recibir un `ModelReport`
- renderizar a distintos formatos mediante templates/renderers (`PDFTemplate`, `CSVTemplate`, futuro `HtmlTemplate`, futuro `XlsxTemplate`)

InformesEstadísticos

Sería la **capa de aplicación**:

- UI de filtros
- selección de formato
- lógica estadística
- construcción del modelo concreto
- invocación a ExtendedReport con:
 - modelo
 - template
 - formato de salida

Y eso me parece una separación muy buena.

Qué implica esto

Que **no necesitas convertir ExtendedReport en algo visualmente complejo por sí mismo.**

Lo que necesitas es que ExtendedReport sea capaz de devolver más tipos de salida.

Ahora mismo ya hace algo parecido:

- `PDFTemplate` → salida PDF
- `CSVTemplate` → salida texto CSV

Así que la evolución natural sería:

- `HtmlTemplate`
- más adelante `XlsxTemplate`

Sin mover la responsabilidad de filtros, formularios y UX fuera de `InformesEstadisticos`.

Lo importante de diseño

Yo mantendría esta regla:

ExtendedReport no sabe nada de filtros ni de pantallas de usuario.

Solo sabe:

- “tengo una estructura”
- “tengo unos datos”
- “los renderizo”

Eso lo mantiene limpio.

Luego `InformesEstadisticos` decide si quiere:

- mostrar HTML en pantalla
- descargar PDF
- exportar CSV
- exportar XLSX

Sobre `HtmlTemplate`

Aquí lo interesante es que no sería una “vista de aplicación”, sino un **renderer de informe**.

O sea:

- `InformesEstadisticos` recoge filtros
- crea el modelo
- lo carga en `ExtendedReport`
- `ExtendedReport` devuelve HTML
- `InformesEstadisticos` lo incrusta en su página

Así respetas el reparto de responsabilidades.

Lo que yo haría

En ExtendedReport

Añadiría una capa de selección de renderer, algo conceptual como:

- pdf
- csv
- html
- xlsx

Y que todos compartan:

- loadTemplate()
- addDataset()
- render()

Ya estás muy cerca de eso con ExportTemplate.

En InformesEstadisticos

Mantendría:

- filtros
- formularios
- controladores
- construcción del modelo estadístico
- llamada final al renderer adecuado

Ventaja de este enfoque

Te permite que InformesEstadisticos siga siendo el plugin “rico” y de negocio, mientras ExtendedReport sigue siendo el motor reutilizable.

Dicho de otro modo:

- **ExtendedReport = motor**
- **InformesEstadisticos = producto que usa el motor**

Y eso es justo lo correcto.

Mi conclusión

Sí, con esa estructura **no intentaría meter la lógica visual de usuario dentro de ExtendedReport.**

Haría esto:

- ExtendedReport evoluciona a motor multi-salida
- InformesEstadisticos sigue controlando filtros, UX y flujo
- HtmlTemplate sería solo una nueva salida del motor, pensada para verse en pantalla

Eso además te deja un camino muy limpio para XLS/XLSX después.

El siguiente paso útil sería definir la interfaz de ese renderer HTML: qué recibe, qué devuelve y cómo traduce `header/detail/footer/group` a bloques HTML sin depender de coordenadas.

Pensó durante un par de segundos

Sí, **seguramente habrá que tocar los widgets**, pero yo intentaría que fuese **lo mínimo imprescindible**.

El punto clave es este:

El XML ahora mezcla dos cosas

1. **estructura lógica**
 - grupo
 - header
 - detail
 - footer
 - columnas
 - fieldname
2. **presentación física PDF**
 - `posx`
 - `posy`
 - `height`
 - anchuras pensadas para papel
 - estilos muy limitados orientados a `Cezpdf`

Para HTML lo primero te sirve mucho.

Lo segundo, solo en parte.

Mi respuesta corta

No haría widgets nuevos desde cero.

Haría que los widgets soporten **otro modo de render**.

Ahora mismo cada widget sabe pintar en PDF.

Por ejemplo `WidgetItem` obliga a esto:

```
abstract public function render(Cezpdf $pdf, float $posX,  
float $posY, float $width, float $height);
```

Eso ya te dice que el diseño actual está **acoplado a PDF**.

Qué cambiaría realmente

Opción buena

Separar en cada widget dos responsabilidades:

- **obtener/preparar el valor**
- **representarlo**

Porque eso ya casi lo tienes:

- `setValue()`
- `getValue()`
- `getFieldName()`
- formateo en `WidgetLabel` y `WidgetNumber`

Entonces el widget no debería saber solo “dibujar PDF”, sino también devolver algo como:

- texto plano
- valor formateado
- clase CSS
- estilo básico
- tipo semántico

Lo que yo haría

1. Mantener `render()` para PDF

Para no romper nada.

2. Añadir algo tipo:

- `renderHtml(): string`
o mejor
- `getHtmlValue(): string`
- `getHtmlMeta(): array`

Por ejemplo:

- `WidgetLabel` → devuelve texto con clases de alineación, bold, underline, color...
- `WidgetNumber` → devuelve número ya formateado, clase para negativos, alineación derecha...
- `WidgetCalculated` → igual que number, pero calculado

Así reutilizas la lógica de valor y formato sin duplicar demasiado.

Qué widgets tocaría seguro

`WidgetItem`

Es la base. Aquí haría el cambio de diseño principal.

`WidgetLabel`

Porque ya contiene:

- `bold`
- `italic`
- `underline`
- `align`
- `color`
- `bgcolor`
- `size`

Eso en HTML se puede traducir muy bien a:

- clases CSS
- estilos inline controlados

WidgetNumber

Porque ya añade:

- formato numérico
- símbolo divisa
- color negativo
- iconos izquierda/derecha

Muy útil en pantalla.

WidgetCalculated

No tanto por HTML, sino porque probablemente querrás mostrar subtotales/totales igual de bien en web.

Lo que no intentaría

No intentaría convertir literalmente esto:

- coordenadas PDF
- rectángulos
- posiciones absolutas

en HTML.

Porque te quedará artificial y feo.

Mejor enfoque

Interpretar el XML así:

- **header** → bloque superior / cabecera de informe
- **detail** → tabla
- **group header** → fila separadora o subtítulo de grupo
- **footer** → fila de total o bloque resumen

Y dentro de cada fila, las `column` serían celdas HTML.

O sea, usar el XML como **estructura del contenido**, no como layout físico.

Entonces, ¿hay que tocar también bandas y columnas?

Sí, un poco.

No solo widgets.

BandItem

Ahora hace esto:

```
$column->render($pdf, $default, $data, $linePos);
```

Para HTML necesitarás algo equivalente tipo:

- `values()` ya te ayuda bastante
- pero seguramente necesitarás algo como `htmlValues()` o `renderHtmlRow()`

ColumnItem

No has pasado ese archivo, pero casi seguro que ahí está el punto donde se mezclan:

- posición
- ancho
- widget

Y ahí será importante decidir qué aprovechas para HTML:

- quizás `width`
- quizás alineación
- pero no `posx/posy`

Mi recomendación concreta

Yo lo haría en dos fases:

Fase 1: HTML funcional y bonito sin obsesionarse con replicar PDF

- reutilizar widgets actuales
- añadir métodos HTML básicos
- ignorar coordenadas absolutas
- representar detail como tabla responsive
- headers/footers como bloques o filas destacadas

Fase 2: enriquecer el XML si hace falta

Solo si ves que falta expresividad, añadir atributos nuevos orientados a pantalla, por ejemplo:

- `class`
- `th`
- `hideonview`
- `hideonpdf`
- `responsive`
- `sortable`

Pero eso después.

Conclusión

Sí, **habrá que tocar widgets**, pero el cambio importante no es “hacer widgets más bonitos”, sino **desacoplarlos de PDF**.

Yo apuntaría a esto:

- los widgets **formatean valores**
- los renderers deciden **cómo se muestran**
- PDF usa coordenadas
- HTML usa tablas/bloques/CSS

Ese reparto te deja un sistema mucho más limpio.

Y sinceramente, creo que ese es el siguiente paso correcto para ExtendedReport.

Si quieres, el siguiente mensaje lo dedicamos a aterrizarlo con una propuesta concreta de clases: `HtmlTemplate`, qué métodos nuevos añadir a `WidgetItem` y cómo traducir `header/` `detail/footer` a HTML sin romper lo actual.

pero el xml lleva también cosas como `translate` (si se traduce o no) si es un label fijo o un campo, negrita, cursiva, colores, color de fondo, líneas horizontales, es decir muchas cosas aunque quizás no te he mostrado un informe completo sólo la estructura base porque sólo te he subido de `ExtendedController` no he subido nada de `InformesEstadísticos`. Te paso un ejemplo. Bueno dos, uno con subgrupos y otro más sencillo

Pensó durante un par de segundos

Sí, con estos ejemplos ya se ve mucho mejor. Y cambia bastante mi matiz: **el XML no solo define estructura**, también lleva bastante **semántica visual reutilizable**.

Así que la respuesta pasa de “habrá que tocar widgets” a algo más preciso:

No tocaría los widgets para rehacerlos, sino para exponer mejor su metadata

Porque ya contienen casi todo lo necesario para una vista HTML decente:

- texto fijo vs campo (`value / fieldname`)
- traducción (`translate`)

- negrita, cursiva, subrayado (**bold**, *italic*, underline)
- color y fondo (**color**, **bgcolor**)
- alineación (**align**)
- formato numérico, moneda, iconos, color negativo
- líneas y separadores, según tus XML (**widget type="line"**)

O sea: **ya tienes una capa de presentación semántica**, aunque ahora esté orientada a PDF.

Lo que no trasladaría tal cual a HTML

Lo que sí es claramente PDF-dependiente es:

- **posx**
- **posy**
- alturas absolutas
- ancho pensado en puntos de papel
- paginación física

Por ejemplo en los XML el layout se apoya mucho en coordenadas exactas.

En pantalla eso no conviene replicarlo 1:1.

En HTML quedaría mejor interpretar eso de forma más flexible:

- **header** principal → bloque superior
- **detail** → tabla
- **group header** → fila/separador de grupo
- **footer** → fila de subtotal/total

Con estos XML veo claro que sí puedes hacer algo “bonito” sin reinventar el sistema

Caso sencillo: **CustomerAnnual**

Aquí prácticamente pide una tabla HTML:

- cabecera superior con empresa, fecha, hora, página/título
- una tabla con columnas:
 - cliente
 - nombre
 - anterior
 - actual
 - diferencia
 - variación
- un footer con totales

Eso está casi ya descrito en XML.

Caso con subgrupo: **ProductBiAnnualGrouped**

Aquí la vista HTML podría ser:

- cabecera general del informe
- para cada **parent**:
 - bloque encabezado del subgrupo
 - tabla con sus filas detalle
 - fila de subtotal
- al final total general

Y eso también está ya bien marcado en XML con `<group name="parent">`, `detail` `fieldname="parentname"` y sus footers.

Entonces, ¿qué tocaría exactamente?

1. Widgets: sí, pero poco

Ahora `WidgetItem` está acoplado a render PDF porque su contrato principal es:

```
abstract public function render(Cezpdf $pdf, float $posX,
float $posY, float $width, float $height);
```

Yo no rompería eso. Añadiría algo paralelo, por ejemplo:

- `getHtmlAttributes(): array`
- `getHtmlClasses(): array`
- `getFormattedValue(): string`

o algo parecido.

Porque el valor ya lo saben calcular/formatear:

- `WidgetItem::setValue() / getValueForFieldName()`
- `WidgetLabel::getValue()` y estilos básicos
- `WidgetNumber::getValue()` con separadores, moneda, negativos, %, etc.

2. ColumnItem/BandItem

Aquí seguramente está el verdadero punto importante.

`BandItem` hoy renderiza recorriendo columnas y delegando en PDF.

Para HTML necesitarás otra lectura de esas columnas:

- no por coordenada absoluta
- sino como “celdas” o “bloques”

Con estos XML, el `detail` encaja casi perfecto como tabla.

3. Nuevo renderer HTML

Un `HtmlTemplate` que:

- lea la misma estructura XML
- pida al widget el valor ya formateado
- convierta bandas en bloques HTML
- use CSS común para que quede bonito

Qué aprovecharía del XML en HTML

Esto sí lo reutilizaría:

- `translate`
- `fieldname / value`
- `bold, italic, underline`
- `align`
- `color, bgcolor`
- `negative`
- `currency`
- `decimal`
- `licon, ricon`
- grupos, subgrupos, footers, subtotales

Esto no lo usaría igual:

- `posx, posy`
- `height` exacta
- lógica de salto de página
- quizá `width` solo como pista, no como mandato

Mi conclusión más afinada con estos ejemplos

No, el problema no es que “falte información visual” en el XML.

De hecho, **hay bastante información reutilizable**.

El problema real es otro:

ahora toda esa información visual está pensada para dibujar en PDF, no para representarse semánticamente en HTML.

Por eso yo haría esto:

- mantener widgets
- añadirles una salida o metadata HTML
- crear un renderer HTML que interprete el XML como estructura lógica
- usar CSS para que se vea bien en pantalla

O sea, el cambio no sería “rediseñar informes”, sino **darle una segunda vida al XML**.

Y sinceramente, viendo `CustomerAnnual.xml` y `ProductBiAnnualGrouped.xml`, creo que es bastante viable.

El siguiente paso útil sería revisar `ColumnItem`, porque ahí probablemente está la clave para traducir columnas PDF a columnas HTML.

Pensó durante un par de segundos

Sí, **esa vía es totalmente válida** y además encaja muy bien con lo que ya tienes.

De hecho, entre las opciones posibles, esta me parece de las más pragmáticas:

- no rompes `ExtendedReport`
- reutilizas XML, bands y widgets
- separas claramente PDF y HTML
- te permite hacer un `HtmlTemplate` bastante libre

La idea base me gusta

Algo así:

- `BandItem::render()` → PDF
- `BandItem::renderHtml()` → HTML
- `WidgetItem::render()` → PDF
- `WidgetItem::renderHtml()` → HTML

y luego:

- `PDFTemplate` sigue con su lógica actual
- `HtmlTemplate` construye HTML con otra estrategia

Eso tiene sentido porque el problema no está en los datos ni en la estructura, sino en el **modo de representación**.

Lo bueno de este enfoque

1. Reaprovechas la semántica actual

Tus widgets ya saben cosas importantes:

- si usan `fieldname` o `value`
- si traducen
- si van en negrita/cursiva
- color y fondo
- formato numérico
- moneda
- color negativo

Eso sirve tanto para PDF como para HTML.

2. El `HtmlTemplate` puede ser muy distinto internamente

Y eso es bueno.

Porque no deberías intentar que `HtmlTemplate` imite el flujo de `PDFTemplate`.

`PDFTemplate` piensa en:

- posiciones
- alturas
- saltos de página
- espacio restante

HtmlTemplate debería pensar en:

- bloques
- tablas
- cabeceras
- subgrupos
- filas de subtotal
- clases CSS

O sea, misma estructura base, distinto render.

3. Mantienes una API mental sencilla

Eso para ti es cómodo:

- cada band sabe pintarse en PDF y HTML
- cada widget sabe pintarse en PDF y HTML

No introduces demasiadas capas raras.

Pero te diría una cosa importante

Yo **no** pondría toda la inteligencia HTML dentro de los widgets.

Es decir, sí a `renderHtml ( )`, pero con cuidado.

Porque si no corres el riesgo de mezclar demasiado:

- el widget decide valor y estilo
- la band decide layout
- el template decide estructura general

Y en HTML eso debería estar más repartido.

Cómo lo haría para que no se ensucie

Widgets

Que `renderHtml ( )` devuelva **el contenido de una celda o bloque**, no una estructura entera complicada.

Por ejemplo:

- `WidgetLabel` → `<span class="...">texto</span>`
- `WidgetNumber` → `<span class="num negative">...</span>`
- `WidgetLine` → `<div class="er-line ..."></div>`

O incluso mejor:

- que devuelva solo el fragmento interno con clases/estilo básicos

Bands

Aquí sí pondría la lógica de composición HTML:

- `BandHeader::renderHtml()` → bloque header
- `BandDetail::renderHtml()` → fila `<tr>` o bloque equivalente
- `BandFooter::renderHtml()` → fila de total / resumen

HtmlTemplate

Aquí estaría la orquestación:

- abrir contenedor del informe
- renderizar cabecera principal
- recorrer dataset
- detectar rupturas
- abrir/cerrar tablas o secciones de grupo
- renderizar subtotales y total general

El punto clave: en HTML no copies el layout PDF

Esto es lo más importante.

Aunque tengas `renderHtml()`, yo **no intentaría usar `posx` y `posy` como base real del layout web**.

Porque en pantalla quieres algo:

- limpio
- responsive
- legible
- mantenible

Entonces `renderHtml()` debe interpretar el XML **semánticamente**.

Ejemplo con tus informes:

CustomerAnnual

- header → bloque superior
- detail → tabla principal
- footer → fila final de totales

ProductBiAnnualGrouped

- header principal → bloque superior
- group **parent** header → cabecera de sección
- detail → tabla del grupo
- footer del subgrupo → subtotal
- footer general → total final

Eso es mucho más natural que intentar posicionar todo en absoluto.

Riesgos de esta solución

1. Duplicación

Tendrás dos renders:

- PDF
- HTML

Sí, habrá algo de duplicidad.

Pero en este caso me parece asumible, porque el objetivo visual es distinto.

2. Algunas bands/widgets tendrán HTML muy simple y otras más complejas

Eso está bien, pero conviene mantener disciplina:

- el widget no construye media página
- la band no formatea datos que pertenecen al widget
- el template no entra al detalle de estilos individuales

3. Si más adelante quieres Twig, podrías replantearlo

Pero ahora mismo no lo veo problema.

Un `HtmlTemplate` que genere HTML “a medida” directamente desde PHP puede ser una fase muy buena.

Mi recomendación concreta

Sí, iría por ahí.

Haría esto:

En widgets

Añadir:

- `renderHtml(...)`
o mejor aún:
- `getHtml(...)`

En bands

Añadir:

- `renderHtml(...)`

Nuevo renderer

- `HtmlTemplate extends ExportTemplate`

En HTML

Usar:

- contenedores
- tablas
- filas de grupo
- bloques de subtotal
- clases CSS comunes del plugin

Un pequeño matiz de diseño

Si quieres que quede limpio, yo casi prefiero esta idea:

- `render()` para PDF
- `toHtml()` para HTML

Solo por semántica, para no mezclar ambos mundos bajo el mismo nombre.

Pero eso ya es gusto de diseño.

La idea de fondo es la misma.

Conclusión

Sí, me parece una muy buena solución.

No solo es viable, sino que probablemente es la forma más natural de evolucionar ExtendedReport sin romper lo que ya funciona.

La clave sería:

- mismo XML
- mismas bands/widgets
- doble salida
- `HtmlTemplate` con lógica propia
- HTML interpretado de forma semántica, no geométrica

Y sí, con eso puedes hacer un HTML bastante a medida y bastante bonito.

El siguiente punto delicado ya no es conceptual, sino de diseño interno: **decidir qué parte del HTML genera cada capa** para que no quede todo demasiado acoplado.

Pensó durante un par de segundos

Sí, te sigo. Y **sí, es compatible con Twig**, y en tu caso creo que **es mejor apoyarte en Twig que meter HTML largo dentro del PHP**.

Mi opinión

Para este caso haría una solución híbrida:

- **PHP** para:
 - preparar datos
 - decidir qué partial Twig usar
 - pasar contexto

- resolver lógica de render del informe
- **Twig** para:
 - pintar el HTML de bands
 - pintar el HTML de widgets
 - mantener el marcado limpio y editable

Por qué lo veo mejor que HTML embebido en PHP

Si metes el HTML en el propio PHP:

- se ensucia rápido
- cuesta más iterar en la vista
- mezclarás estructura HTML con lógica de datos
- luego mantener estilos, tablas, clases y variantes será más incómodo

En cambio con Twig:

- puedes tener parciales por tipo
- se entiende mejor visualmente
- es más fácil retocar el aspecto
- encaja mejor con una futura vista en pantalla “bonita”

Lo que yo haría

Para las bands

Sí, algo como:

- `BandHeader.html.twig`
- `BandDetail.html.twig`
- `BandFooter.html.twig`
- quizá `GroupHeader.html.twig` si lo necesitas

Para los widgets

Aquí tengo más dudas sobre hacerlo 100% por archivo Twig individual.

Porque si haces un Twig por widget:

- `WidgetLabel.html.twig`
- `WidgetNumber.html.twig`
- `WidgetCalculated.html.twig`
- `WidgetLine.html.twig`

puede quedar bien, pero también puedes acabar con demasiada fragmentación.

Mi propuesta concreta

Opción que más me gusta

- **Bands en Twig**
- **Widgets preparan contexto desde PHP**
- los widgets pueden:

- devolver el valor formateado
- devolver clases CSS
- devolver estilos
- devolver tipo semántico

Y luego la band Twig decide cómo pintarlo.

Por ejemplo, un `BandDetail.html.twig` recibe una lista de columnas ya resueltas y hace algo tipo:

- si es tabla, `<tr><td>...</td></tr>`
- si es header visual, `<div class="...">...</div>`

Así evitas que cada widget tenga que montar su propia mini plantilla completa.

Cuándo sí usaría Twig por widget

Solo en widgets que tengan una representación claramente distinta, por ejemplo:

- `line`
- alguno futuro más complejo
- un widget que pinte badge, barra, icono, bloque especial...

Pero para `label`, `number`, `calculated`, muchas veces bastaría con un mismo partial genérico de celda, cambiando clases y contenido.

Diseño que veo más limpio

En PHP

Cada widget expone algo como:

- contenido HTML seguro o texto formateado
- clases
- estilos
- alineación
- tipo

Cada band compone una estructura con columnas ya resueltas.

En Twig

- un template de band pinta la estructura
- quizá un partial común de columna/celda
- y widgets especiales, si hace falta, con partial propio

Respuesta corta a tu duda

Sí, te entiendo, y yo tiraría por Twig.

No metería el HTML grande en el mismo PHP salvo fragmentos muy pequeños.

Porque aquí lo que quieres construir es una **capa de vista HTML** del informe, y ahí Twig te da mucha más limpieza.

Mi recomendación final

Haría esto:

- `HtmlTemplate` en PHP como orquestador
- `partials` Twig para bands
- un `partial` Twig genérico para columnas/widgets simples
- lógica de formato en PHP
- lógica de marcado en Twig

Así aprovechas lo mejor de ambos mundos.

El HTML en PHP solo lo dejaría para casos muy pequeños o utilitarios, no como estrategia principal.

Sí, la conozco 👍

Y justo en este contexto **te viene perfecta como puente entre PHP y Twig**.

Qué te aporta `Html` de FacturaScripts

La clase `Html` del core básicamente te permite:

- renderizar Twig desde PHP
- pasar variables (`fsC`, `dataset`, etc.)
- reutilizar el sistema de plantillas de FS
- generar HTML ya integrado con el entorno

Es decir, te evita tener que montar tú:

- entorno Twig
- loaders
- configuración

Cómo encaja con lo que estás planteando

Tu idea sería algo así (conceptualmente):

- `HtmlTemplate` (PHP)
 - recorre bands
 - prepara datos
 - llama a Twig

Y en vez de hacer:

```
return '<tr><td>...</td></tr>';
```

harías algo tipo:

```
Html::render('ExtendedReport/BandDetail.html.twig', $data);
```

👉 Eso es exactamente el uso correcto.

Dónde veo el mayor valor

1. Bands en Twig

Aquí sí lo usaría sin dudar:

- `BandHeader.html.twig`
- `BandDetail.html.twig`
- `BandFooter.html.twig`

Porque ahí tienes estructura:

- tablas
- filas
- bloques
- agrupaciones

Y Twig lo hace mucho más limpio.

2. Widgets → aquí está la decisión clave

Tienes dos caminos:

Opción A (mi recomendación)

Widgets **NO** renderizan Twig directamente

Solo devuelven:

- valor formateado
- clases
- estilos
- tipo

Y Twig pinta:

```
<td class="{{ col.class }}" style="{{ col.style }}">
    {{ col.value|raw }}
</td>
```

- ✓ más limpio
- ✓ menos fragmentación
- ✓ mejor rendimiento
- ✓ más control desde band/template

Opción B (más flexible, más compleja)

Cada widget llama a su propio Twig:

```
Html::render('WidgetNumber.html.twig', $data);
```

- ✓ más modular
- ✗ más overhead
- ✗ más difícil de mantener a largo plazo
- ✗ puede acabar muy fragmentado

Mi recomendación clara

Conociendo tu proyecto:

- 👉 Usaría **Html.php + Twig para bands**
- 👉 **NO** usaría Twig por cada widget (salvo casos especiales)

Arquitectura que te quedaría

PHP (ExtendedReport)

- `HtmlTemplate`
 - orquesta todo
 - llama a Twig por band
- `BandItem::renderHtml()`
 - prepara estructura de columnas
 - no genera HTML directamente
- `WidgetItem`
 - formatea valor
 - aporta estilos/clases
 - NO decide estructura HTML

Twig

- `BandHeader.html.twig`
- `BandDetail.html.twig`
- `BandFooter.html.twig`
- (opcional) `_cell.html.twig`

Ejemplo mental simple

PHP prepara:

```
[
  'columns' => [
    [
      'value' => 'Cliente 1',
      'class' => 'text-start',
      'style' => '',
    ],
    [
      'value' => '123,45 €',
      'class' => 'text-end text-danger',
      'style' => '',
    ]
  ]
]
```

Twig pinta:

```
<tr>
  {% for col in columns %}
    <td class="{{ col.class }}" style="{{ col.style }}">
      {{ col.value|raw }}
    </td>
  {% endfor %}
</tr>
```

Conclusión

Sí, usar `Html` del core es justo lo que necesitas.

Y la clave sería:

- **Twig para estructura (bands)**
- **PHP para lógica (widgets + datos)**
- evitar meter HTML grande en PHP
- evitar micro-templates por widget salvo excepciones

Con eso te queda un sistema muy limpio, escalable y alineado con FacturaScripts.

Pensó durante un par de segundos

Sí, ahora te sigo mejor. Y me parece una buena idea.

En **PDFTemplate** tiene sentido que el template lleve el control fuerte del render, porque necesita gobernar:

- posición vertical
- espacio restante
- headers/footers
- saltos de página
- ruptura de secuencia

Pero en **HtmlTemplate** ese problema desaparece casi por completo. Ahí puedes permitirte un flujo mucho más declarativo.

Lo que haría yo

En **HtmlTemplate**

Que siga controlando solo la **orquestración**:

- cargar dataset
- recorrer grupos
- detectar rupturas
- decidir cuándo renderizar header/detail/footer
- preparar el contexto para Twig

En las **bands**

Que tengan un `renderHtml()` cuyo trabajo no sea construir todo el HTML a mano, sino **delegar en Twig**.

Algo conceptual:

- `BandHeader::renderHtml($context)`
- `BandDetail::renderHtml($context)`
- `BandFooter::renderHtml($context)`

y dentro:

- preparan sus columnas
- llaman a una plantilla Twig

Eso encaja muy bien con tu idea.

Y sí: las columnas/widgets pueden delegar también

Tu planteamiento de que la **band llame al `renderHtml()` de la columna o widget** me parece razonable, pero con un matiz:

mejor que el widget no pinte una página,

sino solo su fragmento o su contexto

Porque si no puedes acabar con demasiada lógica dispersa.

Yo lo repartiría así:

HtmlTemplate

decide el flujo general

Band...::renderHtml()

llama a Twig de banda y le pasa las columnas ya resueltas

ColumnItem::renderHtml()

prepara:

- ancho relativo o clase
- tipo de contenido
- resultado del widget
- estilos/clases

WidgetItem::renderHtml()

devuelve el fragmento final o al menos el valor formateado para HTML

Ojo con un detalle importante

Aquí veo dos opciones válidas:

Opción A

`BandDetail.html.twig` renderiza directamente cada columna:

```
{% for column in columns %}
    {{ column.rendered|raw }}
{% endfor %}
```

En este caso la columna/widget ya trae el HTML resuelto.

Opción B

`BandDetail.html.twig` recibe datos estructurados:

```
{% for column in columns %}
    <td class="{{ column.class }}"
style="{{ column.style }}">
        {{ column.value|raw }}
    </td>
{% endfor %}
```

Aquí Twig construye más HTML.

Mi preferencia

Para tu caso, haría una mezcla, pero más cerca de la **B**.

Es decir:

- **widget** resuelve valor, formato y quizá clases
- **columna** aporta ancho/clase/metadata
- **band twig** compone la estructura HTML final

Porque así:

- Twig conserva el control visual real
- PHP conserva la lógica de datos/formato
- no embebes demasiado HTML en PHP
- sigues teniendo flexibilidad

En otras palabras

Tu idea de “delegar ciertas tareas al propio Twig” me parece correcta, pero no delegaría **demasiada inteligencia** a Twig.

Twig debería encargarse de:

- estructura
- marcado
- clases
- composición visual

PHP debería encargarse de:

- valor
- traducción
- formato numérico
- color lógico
- detección de tipo de widget
- agrupación y control del flujo

Diseño que veo más limpio

HtmlTemplate

- recorre grupos y filas
- detecta rupturas
- llama a band renderers

BandItem::renderHtml()

- resuelve columnas
- llama a `Html::render()` con su twig

ColumnItem::renderHtmlData()

- devuelve array con info de presentación

WidgetItem::renderHtmlData()

- devuelve algo tipo:
 - `value`
 - `class`
 - `style`
 - `type`

Y luego Twig pinta eso.

Lo bueno de este enfoque

Te separa bastante bien los mundos:

- `PDFTemplate` sigue siendo un renderer técnico y posicional
- `HtmlTemplate` pasa a ser un renderer más declarativo y visual

Sin forzar a que ambos funcionen igual.

Y eso es bueno, porque realmente **no deben funcionar igual**.

Conclusión

Sí, lo veo bien así:

- `HtmlTemplate` controla el flujo general
- las bands tienen `renderHtml()`
- dentro de ese `renderHtml()`, usas Twig
- las columnas/widgets colaboran preparando datos o fragmentos

Yo intentaría que **Twig componga el HTML de la band** y que **PHP prepare el contenido de cada columna/widget**.

Eso te da un equilibrio bastante bueno entre limpieza y control.

El punto siguiente, si quieres, es aterrizar qué debería devolver exactamente

`ColumnItem::renderHtml()` y `WidgetItem::renderHtml()` para que Twig trabaje cómodo.

sí, estamos de acuerdo el widget retorna lo justo para pintar el valor. La idea de `<span>` me parece genial, ya la banda lo meterá en el contenedor que crea (tabla, fila, columna, card, etc)

Pensó durante unos pocos segundos

Sí, justo por ahí lo veo yo.

Reparto limpio

Widget

Debe devolver **solo la pieza mínima**:

- valor ya formateado
- clases
- estilo inline si hace falta
- quizá algún metadato simple (`type`, `align`)

Algo tipo:

```
[
    'tag' => 'span',
    'value' => '123,45 €',
    'class' => 'er-number text-end text-danger fw-bold',
    'style' => 'background-color:#EBEDEF;'
]
```

Eso encaja muy bien con cómo ya tienes la lógica de valor en widgets:

- `WidgetItem` resuelve `fieldname`, arrays, métodos y propiedades anidadas
- `WidgetLabel` ya conoce `align`, `bold`, `italic`, `underline`, `color`, `bgcolor`, `translate`
- `WidgetNumber` ya sabe formatear número, moneda, iconos y color negativo

Band

La band ya compone:

- tabla
- fila
- bloque header
- subtotal
- card
- lo que necesites

Ahí sí tiene sentido que use Twig y meta el resultado del widget dentro de su contenedor.

Ventaja de hacerlo así

Evitas esto:

- widget construyendo `<td>`
- widget decidiendo layout
- widget sabiendo si está en tabla o en card

Y eso es bueno, porque ese trabajo pertenece a la band, no al widget.

En resumen

La regla quedaría así:

- **widget = contenido**
- **columna = metadata de colocación/anchura/clases**

- **band = estructura HTML**
- **HtmlTemplate = orquestación del informe**

Me parece una base muy sólida.

El siguiente punto importante ya sería decidir si `renderHtml()` del widget devuelve:

- un **array de datos**
o
- un **`<span>...</span>` ya montado**

Yo aquí prefiero ligeramente el **array**, porque deja más libertad a Twig.

bueno es tarde, por lo menos para mí que son casi las 2 de la mañana. Pero me ha gustado mucho la charla. Sigo pensando en el tema. Pero me ha gustado mucho el planteamiento

Perfecto, buen momento para parar entonces 😊

La base que has planteado es muy buena y además encaja con lo que ya tienes, sin tirar nada:

- XML sigue siendo la definición
- widgets siguen aportando valor/formato
- bands pasan a estructurar también HTML
- **HtmlTemplate** abre la puerta a vista en pantalla y futuros formatos

Y lo más importante: no estás forzando a que HTML funcione como PDF, sino que cada uno juega con sus reglas.

Cuando lo retomes, seguramente el siguiente paso natural será hacer un **primer HtmlTemplate muy simple (aunque sea feo)** solo para validar flujo:

- render header
- render una tabla básica del detail
- ignorar estilos complejos al principio

Y a partir de ahí ya iterar.

Buen descanso 👍